# Day 97: The Week 14 Build — Language-Model Evaluation: Perplexity, BLEU and ROUGE

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 14, Day 7 | Complete study guide

Watch: Measuring LLM Accuracy with BLEU and ROUGE score (DATAJARVIS) | Language Model Evaluation and Perplexity (Machine Learning TV)

Script: `day97_lm_evaluation.py` · every number in this guide is copied by program from `day97_run_output.txt`, the real output of that script.

The week's six exercise parts, copied verbatim, plus BLEU and ROUGE written from scratch, evaluate four models of a play none of them pretrained on. Perplexity separates them clearly: Day 92's GPT 2.486 bits per character, Day 96's fine-tuned GPT 2.104. BLEU and ROUGE against the real continuation cannot: every model scores BLEU-4 0.006, and a passage of real Shakespeare taken from the wrong place scores as high or higher (ROUGE-1 0.160). During fine-tuning, dev perplexity falls at every check while BLEU wanders with no trend. For open-ended text, reference-overlap metrics measure shared common words, not quality.

## 0. Where today fits

**DEFINITION — start here, no prior knowledge assumed**

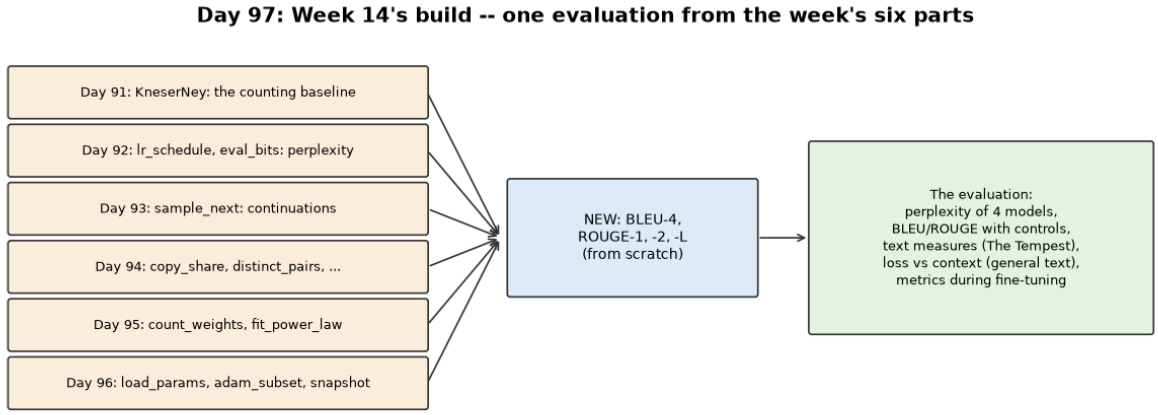
**Perplexity** scores a model by how much probability it gives to real held-out text: $2^{bits}$, the number of equally likely choices the model is effectively choosing between at each step. It needs the model's probabilities, not its outputs.

**BLEU** and **ROUGE** score a piece of *generated* text by its overlap with a human-written reference: BLEU by how many of the generated word sequences appear in the reference (precision, from machine translation), ROUGE by how many of the reference's appear in the generated text (recall, from summarisation).

**REAL-WORLD SCENARIO**

Marking a translation by counting the words it shares with the teacher's answer works when there is one right answer. Marking a short story that way does not: a good story shares few words with any particular other story.

**SYSTEM DIAGRAM**




*The six exercise parts and the new metrics, assembled into one evaluation.*

### The build

Like Days 83 and 90, Day 97 copies each exercise script's functions verbatim, under a banner naming its source: Kneser-Ney (Day 91), the schedule and full-context measure (92), the sampler (93), the text measures (94), the weight count and power-law fit (95), and the model-file and fine-tuning helpers (96). Day 90's GPT is imported. New today: BLEU-4, ROUGE-1, ROUGE-2 and ROUGE-L, and the experiments.

### How to read each section

| Step | What it gives you |
|---|---|
| DEFINITION | What the idea is, in plain words. Start here. |
| REAL-WORLD SCENARIO | An everyday situation where the same idea shows up. |
| MATHEMATICS | The formula, when the section introduces one. |
| SPECIFICATION | The exact setup of the experiment. |
| Why this test | The question the experiment answers. |
| IMPLEMENTATION | The real code from the script. |
| ACTUAL OUTPUT | What the script really printed, copied unchanged. |
| What it means | How to read the numbers, and what they do and do not show. |
| Why choose this? / Why NOT this? | When the idea is the right tool, and what it costs. |

## 1. The build, and BLEU and ROUGE on cases with known answers

**DEFINITION — start here, no prior knowledge assumed**

Both saved models are loaded with Day 96's `load_params` and counted with Day 95's formula. Then the new metrics are checked on four hypotheses against one reference: identical, a shorter prefix, the same words scrambled, and no shared words.

**REAL-WORLD SCENARIO**

Testing a new ruler on objects of known length before measuring anything unknown.

**MATHEMATICS**

$p_n = \Sigma_g \min(count_{hyp}(g), count_{ref}(g)) / \Sigma_g count_{hyp}(g)$ *(clipped n-gram precision)*

$BLEU = BP \cdot \exp(\tfrac{1}{4} \Sigma_{n=1..4} \log p_n)$, $BP = 1$ *if* $c > r$, *else* $e^{1 - r/c}$ *(c, r: hypothesis and reference lengths)*

*ROUGE-N F1 = 2 R P / (R + P),* $R$ *= shared n-grams / reference n-grams,* $P$ *= shared / hypothesis n-grams*

*ROUGE-L: the same F1 with the longest common subsequence of words as the shared part*

**IMPLEMENTATION**

```
def bleu(hyps, refs, max_n=4):
    match, total = np.zeros(max_n), np.zeros(max_n)
    hyp_len = ref_len = 0
    for h, r in zip(hyps, refs):
        hw, rw = words(h), words(r); hyp_len += len(hw); ref_len += len(rw)
        for n in range(1, max_n + 1):
            hc, rc = ngrams(hw, n), ngrams(rw, n)
            match[n - 1] += sum(min(c, rc[g]) for g, c in hc.items())
            total[n - 1] += max(0, len(hw) - n + 1)
    if match[0] == 0:
        return 0.0, match / np.maximum(total, 1)
    prec = np.r_[match[0] / total[0], (match[1:] + 1) / (total[1:] + 1)]
    bp = 1.0 if hyp_len > ref_len else np.exp(1 - ref_len / max(hyp_len, 1))
    return bp * np.exp(np.mean(np.log(prec))), match / np.maximum(total, 1)
```

**ACTUAL OUTPUT**

```
Day 92 (general): 824129 weights loaded from day92_mini_gpt.npz; Day 95's formula gives 824129
Day 96 (fine-tuned): 824129 weights loaded from day96_finetuned_gpt.npz; Day 95's formula gives 824129
largest change made by fine-tuning: 0.026 (in Wout); arrays unchanged: 0 of 52
BLEU and ROUGE on known cases (hypothesis | reference):
  'the king is come to the island'     BLEU 1.000  ROUGE-1 1.000  ROUGE-2 1.000  ROUGE-L 1.000
  'the king is come'                   BLEU 0.472  ROUGE-1 0.727  ROUGE-2 0.667  ROUGE-L 0.727
  'island the to come is king the'     BLEU 0.263  ROUGE-1 1.000  ROUGE-2 0.000  ROUGE-L 0.429
  'my mistress waits below'            BLEU 0.000  ROUGE-1 0.000  ROUGE-2 0.000  ROUGE-L 0.000
Actual output from this lesson's run.
```

### What it means

The parts fit: both files load, and Day 95's formula gives exactly the 824129 weights counted from them. Fine-tuning moved every one of the 52 arrays (0 of 52 unchanged), by at most 0.026. The metrics behave as designed: identical text scores 1.000 everywhere, no shared word scores 0.000. The short prefix is penalised by BLEU's brevity penalty (0.472) and by ROUGE's recall (0.727). The scrambled sentence shows what each one sees: ROUGE-1 is still 1.000 (single words do not care about order), ROUGE-2 drops to 0 and ROUGE-L to 0.429.

**WHY CHOOSE THIS?**

Cheap, deterministic, needs no model: any two texts can be compared.

**WHY NOT THIS? (TRADEOFFS)**

Only exact word matches count: a synonym or a paraphrase scores the same as a wrong word.

## 2. Perplexity: four models of The Tempest's test text

**DEFINITION — start here, no prior knowledge assumed**

Bits per character on the play's 5,199-character test part, for two counting models (Kneser-Ney on the training text, and on the training text plus the play's train part) and two GPTs (Day 92's general model and Day 96's fine-tuned one). Perplexity per character is $2^{\text{bits}}$; per word it is $2^{\text{bits} \times \text{characters per word}}$.

**REAL-WORLD SCENARIO**

Comparing weather forecasters by the probability each gave to the weather that actually happened.

**MATHEMATICS**

$$\text{bits} = -(1/N)\,\Sigma_i \log_2 p(x_i \mid x_{<i}), \quad \text{perplexity per character} = 2^{\text{bits}}, \quad \text{per word} = 2^{\text{bits} \cdot (\text{characters per word})}$$

**IMPLEMENTATION**

```
for name, text in (("Kneser-Ney, training text", tr_text), ("Kneser-Ney, + the play's train part",
    tr_text + ft_tr)):
    kn = KneserNey(text, KN_ORDER, KN_D); kns.append(kn); rows.append((name, kn.bits(ft_te,
        KN_ORDER - 1)))
for name, (P, vocab, cfg) in models.items():
    ix = {c: i for i, c in enumerate(vocab)}
    rows.append((name, eval_bits(gpt_logp(P, cfg), np.array([ix[c] for c in ft_te]),
        cfg["T"])[0]))
print(f"\n{'model':<36}{'bits/char':>10}{'ppl/char':>10}{'ppl/word':>10}")
for name, b in rows:
    print(f"{name:<36}{b:>10.3f}{2 ** b:>10.2f}{2 ** (b * cpw):>10.0f}")
```

**ACTUAL OUTPUT**

```
model                                bits/char  ppl/char  ppl/word
Kneser-Ney, training text                2.460      5.50     15979
Kneser-Ney, + the play's train part      2.142      4.41      4568
Day 92 (general)                         2.486      5.60     17709
Day 96 (fine-tuned)                      2.104      4.30      3928
(5.68 characters per word in the test part; the two kinds of score are on slightly different
    positions)
Actual output from this lesson's run.
```

### What it means

Perplexity ranks the models clearly and in the expected direction: seeing the play's train part helps both kinds of model, Kneser-Ney from 2.460 to 2.142 and the GPT from 2.486 to 2.104. One result is a surprise: on this unseen play the plain counting model (2.460) is slightly ahead of Day 92's GPT, though the GPT was level with it on the general held-out text (Day 92). Per word, the same gap looks huge (17709 against 3928) because bits per character are multiplied by 5.68 characters per word before being raised to a power: per-word perplexities exaggerate small differences and should only be compared on the same text.

**WHY CHOOSE THIS?**

Uses every character of the test text, needs no reference outputs, and separates models reliably.

**WHY NOT THIS? (TRADEOFFS)**

Needs the model's probabilities; models with different vocabularies or tokenisers are not directly comparable.

## 3. BLEU and ROUGE: continuations against what Shakespeare wrote

**DEFINITION — start here, no prior knowledge assumed**

Each system continues 24 prompts of 64 characters from the play's test part by 150 characters; the reference is the real next 150 characters. Three **controls** show what the scale means: the reference itself (the maximum), real Shakespeare from the wrong place (the true continuation of a different prompt), and the prompt repeated.

**REAL-WORLD SCENARIO**

Judging a student's ending to a story by how many words it shares with the author's ending, and checking how a different chapter of the same book would have scored.

**SPECIFICATION**

| | |
|---|---|
| **Prompts** | 24 windows of 64 + 150 characters, evenly spaced, so no two share any text. |
| **Sampling** | top-p 0.9 (Day 93's sampler), seeds 0, 1, 2; mean and spread over seeds. |
| **Systems** | Day 92's GPT, Day 96's GPT, and Kneser-Ney trained on the training text plus the play's train part. |

### Why this test

Section 2 says the fine-tuned model is much better. If BLEU and ROUGE measure quality, they should say so too, and a model should beat real text from the wrong place. A first version drew the prompts at random, so some windows overlapped: the 'wrong place' control then shared real text with its reference and scored several times higher. The run reported here uses non-overlapping windows.

**IMPLEMENTATION**

```
starts = np.arange(N_PROMPTS) * (PROMPT_LEN + CONT_LEN)
prompts = [ft_te[s:s + PROMPT_LEN] for s in starts]
refs = [ft_te[s + PROMPT_LEN:s + PROMPT_LEN + CONT_LEN] for s in starts]
other = refs[1:] + refs[:1]
```

**ACTUAL OUTPUT**

```
system                          BLEU-4       ROUGE-1       ROUGE-2       ROUGE-L
(reference itself)               1.000         1.000         1.000         1.000
(real text, wrong place)         0.008         0.160         0.010         0.104
(prompt repeated)                0.008         0.090         0.008         0.083
Day 92 (general)           0.006 (0.000)   0.133 (0.006)   0.003 (0.001)   0.095 (0.002)
Day 96 (fine-tuned)        0.006 (0.001)   0.129 (0.012)   0.003 (0.002)   0.096 (0.008)
Kneser-Ney (+ play)        0.006 (0.000)   0.129 (0.015)   0.003 (0.001)   0.094 (0.007)
Actual output from this lesson's run.
```

### What it means

**The metrics cannot tell the models apart.** All three systems score BLEU-4 0.006; ROUGE-1 is 0.133, 0.129 and 0.129, differences smaller than the spread between sampling seeds (up to 0.015). The fine-tuned GPT, much better by perplexity, is not better here.

**Real text from the wrong place scores at least as high as any model:** ROUGE-1 0.160 and ROUGE-L 0.104, both above every model, and BLEU-4 0.008. Repeating the prompt also beats every model on BLEU-4 (0.008), though not on ROUGE. On open-ended text these scores most likely count shared common words, which any passage of Shakespeare shares with any other (the shared words were not tallied here). A continuation can be good and still share almost no 4-grams with the one the author happened to write.

**WHY CHOOSE THIS?**

Right for tasks with a narrow set of correct outputs: translation, summaries, short answers.

**WHY NOT THIS? (TRADEOFFS)**

Wrong for open-ended generation, where many different continuations are equally good.

## 4. Day 94's measures on the same continuations

**DEFINITION — start here, no prior knowledge assumed**

Day 94's four text measures on the seed-0 continuations and on the real ones: the share of 16-character pieces copied from anything either model trained on, distinct word pairs, real words, and speaker lines.

**REAL-WORLD SCENARIO**

Checking a forged signature for the habits of the real signer instead of matching it stroke by stroke.

**ACTUAL OUTPUT**

```
text                      copied (k=16)  distinct pairs  real words  speaker lines
(real continuations)              0.013           0.955       0.932          0.320
Day 92 (general)                  0.019           0.966       0.932          0.203
Day 96 (fine-tuned)               0.005           0.972       0.883          0.207
Kneser-Ney (+ play)               0.036           0.698       0.961          0.273
Actual output from this lesson's run.
```

### What it means

These measures do see differences BLEU missed. Kneser-Ney repeats itself (distinct word pairs 0.698 against 0.955 for the real text) and copies the most (0.036). Fine-tuning lowered copying (0.019 to 0.005) and invented more non-words (real words 0.932 to 0.883), a likely effect of its new character names being spelled from too little data (not tested). Both GPTs write fewer speaker lines than the play (0.203 and 0.207 against 0.320). One seed, 24 short texts: small differences here are not reliable.

**WHY CHOOSE THIS?**

Each measure checks one property that matters, with a known value for real text.

**WHY NOT THIS? (TRADEOFFS)**

None of them says whether the text is good; they detect specific failures.

## 5. How the loss falls with context

**DEFINITION — start here, no prior knowledge assumed**

The loss at each position of a window: the prediction at position n has seen n characters. Measured for Day 92's model on the 76,885-character general held-out text (600 windows, far less noisy than the play's test part) and fitted with Day 95's power law from n = 1, 4 and 16 on.

**REAL-WORLD SCENARIO**

How much better you guess the next word of a sentence after reading one word, a few words, or a whole paragraph.

**IMPLEMENTATION**

```
def position_bits(logp_fn, ids_, T):
    n = (len(ids_) - 1) // T
    x, y = ids_[:n * T].reshape(n, T), ids_[1:n * T + 1].reshape(n, T)
    lp = np.concatenate([logp_fn(x[s:s + 32]) for s in range(0, n, 32)])
    return -np.take_along_axis(lp, y[..., None], axis=-1)[..., 0].mean(axis=0) / np.log(2)
```

**ACTUAL OUTPUT**

```
Day 92's model on the rest of the held-out text (76,885 characters; 600 non-overlapping windows of 128): bits per character
    by how many characters the prediction sees, and L(n) = c + a n^-alpha fitted from n = 1, 4 and 16 on.
  1: 3.66, 2: 2.90, 4: 2.58, 8: 2.20, 16: 2.19, 32: 2.10, 64: 2.18, 128: 2.10
  fit from n =  1: c = 1.261, a = 1.270, alpha = 0.115; largest gap 1.127 bits (c searched on [0, 1.829])
  fit from n =  4: c = 0.848, a = 1.424, alpha = 0.041; largest gap 0.380 bits (c searched on [0, 1.829])
  fit from n = 16: c = 0.000, a = 2.011, alpha = -0.004; largest gap 0.212 bits (c searched on [0, 1.829])
  mean bits at n = 1-8: 2.639; n = 9-16: 2.145; n = 17-64: 2.035; n = 65-128: 2.058
Actual output from this lesson's run.
```

### What it means

Almost all the benefit of context comes in the first 16 characters: 3.66 bits with one character, 2.639 on average over the first eight, 2.145 for 9-16, and then 2.035 and 2.058: flat. No single power law describes this. Fitted from the start, it misses the first point by 1.127 bits; from n = 16 on, the best exponent is -0.004 and the floor sits at 0, the edge of the allowed range: there is no decline left to fit. The model's 128-character window is mostly unused, consistent with Day 90's GPT only tying a 4-character counter; why (model size, training length or the text itself) was not tested.

**WHY CHOOSE THIS?**

Shows directly how much context a model uses.

**WHY NOT THIS? (TRADEOFFS)**

One model, one text; the curve is noisy point by point.

## 6. Metrics as a training signal

**DEFINITION — start here, no prior knowledge assumed**

A short fine-tune of Day 92's model (top block and output layer, Day 96's adam_subset, Day 92's schedule), checking dev bits and BLEU-4 of 20 dev continuations every 50 steps, with one fixed sampling seed so BLEU changes only when the model does.

**REAL-WORLD SCENARIO**

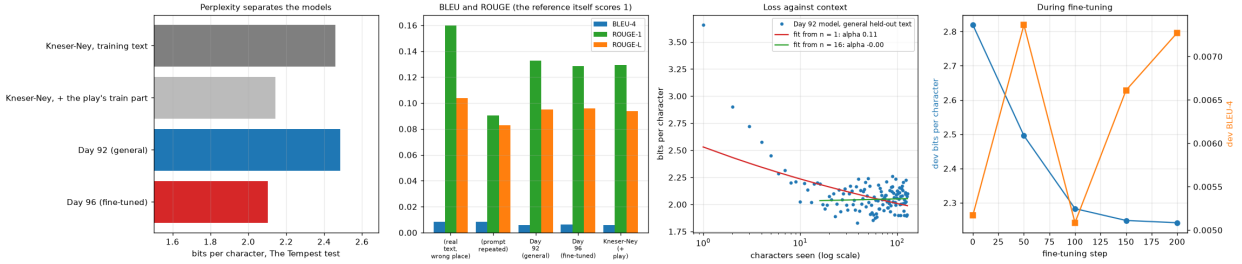
Tracking a runner's training with a stopwatch versus with judges' scores of their running style.

**IMPLEMENTATION**

```
def check():
    hyps = ["".join(vocab[i] for i in c) for c in gpt_continue(P, cfg, prompts, CONT_LEN,
        np.random.RandomState(0), top_p=0.9)]
    return eval_bits(gpt_logp(P, cfg), dev, cfg["T"])[0], bleu(hyps, refs)[0]
...
rng = np.random.RandomState(0); state = {"t": 0, "m": {}, "v": {}}
T, B = cfg["T"], cfg["B"]
rows = [(0,) + check()]; best = (rows[0][1], 0, snapshot(P))
for step in range(1, FT_STEPS + 1):
    o = rng.randint(0, len(tr) - T - 1, B)
    x = np.stack([tr[i:i + T] for i in o]); y = np.stack([tr[i + 1:i + T + 1] for i in o])
    _, G = d90.gpt_loss_grads(x, y, P, h=cfg["h"])
    adam_subset(P, G, state, lr_schedule(step - 1, FT_LR, 10, FT_STEPS), keys)
    if step % FT_CHECK == 0:
        rows.append((step,) + check())
        if rows[-1][1] < best[0]:
            best = (rows[-1][1], step, snapshot(P))
```

**ACTUAL OUTPUT**

```
  step  dev bits  dev BLEU-4
     0     2.819      0.0052
    50     2.496      0.0074
   100     2.283      0.0051
   150     2.248      0.0066
   200     2.242      0.0073
best checkpoint by dev bits: step 200
Actual output from this lesson's run.
```




*Left to right: perplexity of the four models; BLEU and ROUGE with the controls; loss against context with two fits; dev bits and BLEU during fine-tuning (note the BLEU axis covers a tiny range).*

### What it means

Dev bits fall at every check, 2.819 to 2.242. BLEU goes 0.0052, 0.0074, 0.0051, 0.0066, 0.0073: up and down with no trend, at levels where one shared 4-gram moves the score. Choosing a checkpoint by BLEU here would be choosing by noise; perplexity is the signal to train and select on.

**WHY CHOOSE THIS?**

Perplexity is smooth, cheap and tied to the training loss.

**WHY NOT THIS? (TRADEOFFS)**

It does not see sampling choices: a model with good perplexity can still produce dull text with a bad sampler (Day 93).

## Interview preparation — technical questions

**Q: How is perplexity computed?**

A: 2 to the power of the mean negative log2-probability of held-out text; here 2.104 bits per character is 4.30 per character.

**Q: What does BLEU's clipping do?**

A: Counts each n-gram at most as often as in the reference, so repeating 'the the the' cannot score.

**Q: Why does BLEU need smoothing on short texts?**

A: With no shared 4-gram the geometric mean is 0; BLEU+1 adds one to the 2- to 4-gram counts.

**Q: How do ROUGE-1 and ROUGE-L differ?**

A: ROUGE-1 ignores order (the scrambled sentence scored 1.000); ROUGE-L uses the longest in-order match (0.429).

**Q: Why use controls when reporting a metric?**

A: They show what the scale means: here real text from the wrong place beat every model.

## Interview preparation — theoretical questions

**Q: When are BLEU and ROUGE appropriate?**

A: When correct outputs are few and similar: translation, summarisation, short answers.

**Q: Why do they fail for open-ended generation?**

A: Many continuations are equally good, and overlap with any one of them is likely to be mostly common words.

**Q: Why compare perplexity only within one tokeniser?**

A: Bits per character and per word token measure different units; per-word perplexity exaggerates differences.

**Q: What replaces BLEU for open-ended text?**

A: Human judgement, model-based judges, and targeted checks like Day 94's; none is perfect.

## Key takeaway and what's next

> **KEY TAKEAWAY**
> Perplexity and overlap metrics answer different questions. Perplexity separated the models clearly (2.486 against 2.104 bits) and fell smoothly during training. BLEU and ROUGE, built from scratch and checked on known cases, could not separate the models on open-ended continuation, ranked real text from the wrong place above them, and gave no training signal. Week 14 is complete: one mini-GPT, from the chain rule to its evaluation.

- Perplexity on The Tempest's test part: Kneser-Ney 2.460, + the play 2.142; Day 92's GPT 2.486, Day 96's 2.104.
- BLEU-4 0.006 for every system; real text from the wrong place 0.008; ROUGE-1 0.160 for that control, at most 0.133 for a model.
- Context: 3.66 bits after one character, flat after about 16; no power law fits the whole curve.
- During fine-tuning: dev bits 2.819 to 2.242 at every check; BLEU with no trend.

### Suggested watch order, if starting from zero

- 1. Language Model Evaluation and Perplexity (Machine Learning TV) — perplexity and why it measures a language model.
- 2. Measuring LLM Accuracy with BLEU and ROUGE score (DATAJARVIS) — BLEU and ROUGE as used for LLM outputs.

### Day 98 preview

- **Week 15: practical NLP with pretrained models.** A real pretrained transformer library: loading a sentiment model, looking inside its tokenizer and outputs, and measuring it on a real dataset.